

Reviewer Pack — Minimal Rank of Tropicalized Symplectic Geometry over Manifo...

Entry 3487401ccb43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 01:06:06 UTC

Minimal Rank of Tropicalized Symplectic Geometry over Manifolds vs AC0 Circuit Size

Entry ID: 3487401ccb43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 01:06:06 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Symplectic Geometry) **Field B** (complexity object): Complexity Theory: AC0 Circuit Complexity

Statement:

{'text': 'For every compact symplectic manifold M and any subset of points S in M , the minimal rank of the tropicalized symplectic form on M with respect to S is upper-bounded by the size of the smallest AC0 circuit computing the XOR-AND network defined by S .', 'invariant': 'Minimal Rank (Tropicalized Symplectic Geometry)', 'corresponding_complexity_object': 'Size of the smallest AC0 circuit'}

Rationale (proposer's reasoning):

{'text': 'This conjecture proposes a connection between the geometric properties of symplectic manifolds in tropical geometry and their computational complexity. It suggests that the structure preserved by tropicalization might lead to efficient algorithms for certain XOR-AND networks.', 'explanation': 'Symplectic geometry, when treated tropically, could reveal underlying algebraic structures that are relevant to circuit complexity.'}

Taxonomy category: TROPICAL_SYMPLECTIC_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed6b02d96eec339d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the minimal rank values from tropicalized symplectic forms on compact symplectic manifolds is less than or equal to the mean of the smallest AC0 circuit sizes for computing XOR-AND networks, with all seeds having a correlation coefficient ≥ 0.8 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank AND tropical geometry AND symplectic geometry AND AC0 complexity - AC0 circuit size AND related to tropicalized symplectic manifold - tropicalization of symplectic geometry AND bounds on XOR-AND network with AC0 circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for j in range(i + 1, n):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]

def rank(matrix):
    n = len(matrix)
    copy_matrix = [row[:] for row in matrix]
    gaussian_elimination(copy_matrix)
    rank = 0
    for i in range(n):
        if any(copy_matrix[i]):
            rank += 1
    return rank
```

```

def compute_tropicalized_rank(M):
    n = len(M)
    tropicalized_matrix = [[min(M[i][k], M[k][j]) for k in range(n)] for i in range(n)]
    return rank(tropicalized_matrix)

def compute_xor_and_network_size(S):
    n = len(S)
    if n == 1:
        return 1
    else:
        mid = n // 2
        left_size = compute_xor_and_network_size(S[:mid])
        right_size = compute_xor_and_network_size(S[mid:])
        return 1 + left_size + right_size

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random compact symplectic manifold M and subset S
    n = random.randint(5, 40)
    M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    S = [random.sample(range(n), k=random.randint(1, n)) for _ in range(30)]

    tropicalized_ranks = [compute_tropicalized_rank(M) for _ in range(30)]
    xor_and_network_sizes = [compute_xor_and_network_size(s) for s in S]

    if not tropicalized_ranks or not xor_and_network_sizes:
        return {
            "metric_name": "Minimal Rank (Tropicalized Symplectic Geometry)",
            "metric_value": 0,
            "instances_tested": len(tropicalized_ranks),
            "conjecture_holds": False,
            "counterexample": "Empty list of ranks or sizes"
        }

    mean_tropical_rank = sum(tropicalized_ranks) / len(tropicalized_ranks)
    mean_xor_and_network_size = sum(xor_and_network_sizes) / len(xor_and_network_sizes)

    correlation_coefficient = 0
    if mean_tropical_rank != 0 and mean_xor_and_network_size != 0:
        numerator = sum((t - mean_tropical_rank) * (x - mean_xor_and_network_size) for t, x in zip(tropicalized_ranks, xor_and_network_sizes))
        denominator = math.sqrt(sum((t - mean_tropical_rank) ** 2 for t in tropicalized_ranks))
        correlation_coefficient = numerator / denominator

    conjecture_holds = correlation_coefficient >= 0.8
    counterexample = "" if conjecture_holds else "Correlation coefficient < 0.8"

    return {
        "metric_name": "Minimal Rank (Tropicalized Symplectic Geometry)",
        "metric_value": mean_tropical_rank,
        "instances_tested": len(tropicalized_ranks),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction=1")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample='Correlation coefficient < 0.8' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f53e5844.py", line 105, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f53e5844.py", line 67, in run_trial
    tropicalized_ranks = [compute_tropicalized_rank(M) for _ in range(30)]
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f53e5844.py", line 46, in compute_tropicalized_rank
    tropicalized_matrix = [[min(M[i][k], M[k][j]) for k in range(n)] for i in range(n)]
                           ^
NameError: name 'j' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15812
2	preregistration	ollama_remote	glm4:latest	0	0	10952
3	novelty	ollama_remote	glm4:latest	0	0	8421
4	novelty	ollama_remote	glm4:latest	0	0	8904
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18052
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12559
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12107
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12830
9	judge	ollama_remote	glm4:latest	0	0	14908

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 114544 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3487401ccb43.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3487401ccb43.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3487401ccb43.tar.gz` (if generated)